

Computational Thinking & Coding — Week 4 Course Material

Computational Thinking & Coding · Week 4 course material · Seoul Theological University

One 3-hour session per week × 15 weeks · All materials in English · For non-major beginners Source curriculum: MIT OCW 6.100L *Introduction to CS and Programming Using Python* (Fall 2022, Ana Bell) — Lectures 7–8, Recitation 4 **Week 4 scope:** decomposition and abstraction · writing functions · docstrings · functions as objects · lambda · tuples and lists **Bring to class:** the `is_even` code you ran (§3) with your three predictions written down

Learning objectives — what you can do at the end of Week 4

#	Objective
1	Explain abstraction and decomposition in one sentence each, with a non-computer example
2	Write a function with parameters, a <code>return</code> , and a docstring that states input and output
3	Say what happens when a file contains only a <code>def</code> and no call
4	Bind a function to a new name and explain why <code>my_func(4)</code> behaves like <code>is_even(4)</code>
5	Write a lambda and say how it differs from a <code>def</code> function
6	Trace nested loops over two tuples and predict every comparison

1. What we learn this week (note §...)

When you use a smartphone, you don't need to know how its circuits work — you touch the screen and get the result you want. The lecture notes call this a “**black box**.” This week we learn to package code into such black boxes, called **functions**: splitting a big problem into self-contained pieces (**decomposition**) while hiding the tedious details (**abstraction**). We also get a first taste of the idea that **a function is itself an object**, just like a number.

2. Terms to know (note §...)

Term	Meaning
Abstraction	Hiding how something works inside; you only need the relationship between input and output
Decomposition	Splitting code into self-contained, reusable pieces (modules)
Function	A reusable piece of code that captures the steps of a computation
Parameter	An input to a function (a “formal parameter” in the definition)
Return	Giving a value back to the caller, using the keyword <code>return</code>
Docstring	The <code>""" . . . """</code> specification of a function — a “ contract ” between coder and user

3. Live coding — `is_even` (note §...)

This is the `is_even` example from Lecture 7. Type it in and run it.

```
def is_even(i):
    """
    Input: i, a positive int
    Returns True if i is even, otherwise False
    """
    return i % 2 == 0

print(is_even(3))
print(is_even(8))
my_func = is_even      # a function bound to a new name (taste of Lecture 8)
print(my_func(4))
```

★ **Predict before you run** (from the preview sheet):

1. What will each of the three print lines display? Write your guesses first.
2. Why does `my_func(4)` work exactly like `is_even(4)` after the assignment?
3. If the file contained only the `def` block and no call like `is_even(3)`, what would appear on the screen?

Board work — fill in during class:

line	your prediction	what actually printed
<code>print(is_even(3))</code>		
<code>print(is_even(8))</code>		
<code>print(my_func(4))</code>		
file with <code>def</code> only		

4. The docstring is a contract [note §...]

★ This paragraph is **the instructor's framing**, not text from the source sheets. The docstring in §3 says two things: what the caller must supply (`i, a positive int`) and what the caller will get back (`True if i is even, otherwise False`). That is the whole black box seen from outside. Someone using your function should not have to read the body — and if they must, the abstraction has failed.

5. Lab B (recitation-style, 40 min) — ground rule [note §...]

The TA never gives away answers — the TA only asks tracing questions.

Warm-up (about 5 min) — lambda functions (original Problem 1)

Original Problem 1. Fill in the blanks, then predict the two printed values before running: _____ / _____

```
# a) cube root of a given number
f1 = lambda x: x ** (____)      # (1) the exponent that gives a cube root
# b) product of two arguments
f2 = lambda x, y: ____ * ____   # (2)(3) the two parameter names

print(f1(8))
print(f2(4, 5))
```

6. Core exercises (about 30 min) — Exercise 1: count the 1s in a tuple (original Problem 2) [note §...]

(a) **Worked example** — read the original solution, then run the test.

```
def count_number_one(tup):
    count = 0
    for elem in tup:
```

```

    if elem == 1:
        count += 1
    return count

```

```
print(count_number_one((1,2,3,4,5,1,1)))
```

(b) One-line change — change the test tuple to (1,2,3,4,5). Predict the output before running: _____

(c) Fill in the blank — complete the function.

```

count = 0
for elem in ____:
    if ____ == 1:
        count += ____

```

TA tracing questions

1. “Halfway through the test tuple — after elem has been 5 — what is in count?”
2. “What is the difference between the tuple (1,) and the integer (1)?”

7. Exercise 2 — common elements of two tuples (original Problem 3) [note §...

(a) Worked example — read the original solution, then run the test.

```

def common_elements(tup1, tup2):
    output_tuple = ()
    for x in tup1:
        for y in tup2:
            if x == y:
                output_tuple += (x,)
    return output_tuple

```

```
print(common_elements((2,3,4), (3,4,5,6)))
```

(b) One-line change — change the test to common_elements((2,3,4), (5,6,7)). Predict the output before running: _____

(c) Fill in the blank — complete the nested loop.

```

for x in tup1:
    for ____ in tup2:
        if x == y:
            output_tuple += (____,)
    return ____

```

TA tracing questions

1. “When x is 3 for the first time, how many times does the inner loop run — and what is in output_tuple when it finishes?”
2. “Why (x,) with a comma instead of (x) when growing the tuple?”

8. Exercise 3 — keep only sublists inside a range (original Problem 4) [note §...

(a) Worked example — remove sublists containing any element outside a given range.

```

def remove_list_range(input_list, low, high):
    output_list = []
    for i in input_list:
        if min(i) >= low and max(i) <= high:
            output_list.append(i)

```

```
return output_list
```

```
print(remove_list_range([[2], [0], [1, 2, 3], [0, 1, 2, 3, 6, 7], [9, 11], [13, 14, 15, 17]], 13, 17))
```

(b) One-line change — change the call’s range to (0, 7). Predict which sublists survive before running: _____

(c) Fill in the blank — complete the filter condition.

```
for i in input_list:
    if ____ (i) >= low and ____ (i) <= high:
        output_list. ____ (i)
```

TA tracing questions

1. “When i is [9, 11] with range 13–17, what do `min(i)` and `max(i)` return, and which side of the `and` fails?”
2. “What is in `output_list` at this point, right before the loop reaches [13, 14, 15, 17]?”

9. Self-check (about 5 min) [note §...

- I can write a one-line lambda and explain how it differs from a `def` function.
- I can trace nested loops over two tuples and predict every comparison made.
- I can explain why checking `min(i)` and `max(i)` is enough to check every element of a sublist.

10. Questions to think about [note §...

1. What is the difference between **defining** a function (`def`) and **calling** (invoking) it? Explain in your own words: “a function’s code only runs when you call it.”
2. Smartphone component makers build a working phone without knowing each other’s internal designs. What are **two benefits** of splitting a program into functions in the same way?